

INFORMATIQUE 2

Algorithmique 2 / Python

Chapitre 2 :

Fonctions et procédures

Par:

Pr. Mohamed-Amine Chadi

Email: m.chadi@uca.ac.ma

Plan:

1. Motivation (pourquoi les fonctions, principe DRY)
2. Syntaxe (définition et appel)
3. Fonctions et procédures
4. Arguments positionnels et arguments par mot-clé
5. Variables locales et variables globales
6. Exemples et exercices

Motivation

Principe DRY: Don't repeat yourself

► # Imaginez on veut trier les nombres dans les listes suivantes:

```
arr1 = [64, 7, 5, 12, 2, 11, 0]
arr2 = [4, 13, 2, 12, 12, 31, 9]
arr3 = [14, 4, 25, 12, 20, 11, 20]
arr4 = [2, 34, 5, 2, 22, 10, 40]
arr5 = [16, 3, 15, 12, 2, 11, 19]
arr6 = [33, 4, 21, 12, 22, 1, 87]
```

tri à bulle pour la liste arr1

```
n = len(arr1)
for i in range(n):
    for j in range(0, n-i-1):
        if arr1[j] > arr1[j+1]:
            arr1[j], arr1[j+1] = arr1[j+1], arr1[j]
```

arr1

[0, 2, 5, 7, 11, 12, 64]

Copier-coller le même code et changer arr1 par arr2

je dois copier le code et le coller et remplacer arr1 par arr2 dans toutes les places
il se peut que j'oublie une place (résultant donc en une erreur)

```
n = len(arr2)
for i in range(n):
    for j in range(0, n-i-1):
        if arr2[j] > arr2[j+1]:
            arr2[j], arr2[j+1] = arr2[j+1], arr2[j]
```

arr2

[2, 4, 9, 12, 12, 13, 31]

refaire la même chose maintenant pour arr3

et puis arr4

et arr5

et arr6

etc.

Imaginez si vous avez beaucoup de listes (ce qui est le cas dans les applications réelles)
où on traite chaque jours plusieurs données différentes d'utilisateurs différents etc.

la solution pour ce problème est -----> les fonctions

Solution : utiliser les fonctions

Et on l'appelle avec différents paramètres (entrées)

```
arr1 = [64, 7, 5, 12, 2, 11, 0]
arr2 = [4, 13, 2, 12, 12, 31, 9]
arr3 = [14, 4, 25, 12, 20, 11, 20]
arr4 = [2, 34, 5, 2, 22, 10, 40]
arr5 = [16, 3, 15, 12, 2, 11, 19]
arr6 = [33, 4, 21, 12, 22, 1, 87]
```

On écrit la fonction une seule fois

```
def tri_a_bulle(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

```
result = tri_a_bulle(arr1)
result
```

```
[0, 2, 5, 7, 11, 12, 64]
```

```
result = tri_a_bulle(arr2)
result
```

```
[2, 4, 9, 12, 12, 13, 31]
```

```
result = tri_a_bulle(arr3)
result
```

```
[4, 11, 12, 14, 20, 20, 25]
```

```
result = tri_a_bulle(arr4)
result
```

```
[2, 2, 5, 10, 22, 34, 40]
```

```
result = tri_a_bulle(arr5)
result
```

```
[2, 3, 11, 12, 15, 16, 19]
```

```
result = tri_a_bulle(arr6)
result
```

```
[1, 4, 12, 21, 22, 33, 87]
```

ou beaucoup plus mieux

```
# ou bien plus compact --> dans une boucle for:
```

```
listes = [arr1, arr2, arr3, arr4, arr5, arr6]
```

```
for a in listes:
    res = tri_a_bulle(a)
    print(res)
```

```
[0, 2, 5, 7, 11, 12, 64]
[2, 4, 9, 12, 12, 13, 31]
[4, 11, 12, 14, 20, 20, 25]
[2, 2, 5, 10, 22, 34, 40]
[2, 3, 11, 12, 15, 16, 19]
[1, 4, 12, 21, 22, 33, 87]
```

Donc, pourquoi les fonctions ?

- En programmation, les fonctions sont très utiles pour réaliser plusieurs fois la même opération au sein d'un programme. Elles rendent également le code plus lisible et plus clair en le fractionnant en blocs logiques.
- L'utilisation de fonctions favorise la modularité du code, car elles permettent de diviser un programme complexe en parties gérables et de réutiliser des blocs de code dans différentes parties du programme.

```
result = tri_a_bulle(arr1)
result
```

```
[0, 2, 5, 7, 11, 12, 64]
```

```
result = tri_a_bulle(arr2)
result
```

```
[2, 4, 9, 12, 12, 13, 31]
```

```
result = tri_a_bulle(arr3)
result
```

```
[4, 11, 12, 14, 20, 20, 25]
```

```
result = tri_a_bulle(arr4)
result
```

```
[2, 2, 5, 10, 22, 34, 40]
```

```
result = tri_a_bulle(arr5)
result
```

```
[2, 3, 11, 12, 15, 16, 19]
```

```
result = tri_a_bulle(arr6)
result
```

```
[1, 4, 12, 21, 22, 33, 87]
```

ou bien plus compact --> dans une boucle for:

```
listes = [arr1, arr2, arr3, arr4, arr5, arr6]
```

```
for a in listes:
    res = tri_a_bulle(a)
    print(res)
```

```
[0, 2, 5, 7, 11, 12, 64]
[2, 4, 9, 12, 12, 13, 31]
[4, 11, 12, 14, 20, 20, 25]
[2, 2, 5, 10, 22, 34, 40]
[2, 3, 11, 12, 15, 16, 19]
[1, 4, 12, 21, 22, 33, 87]
```

Syntaxe

```
def nomDeLaFonction (arg_1, arg_2, arg_3):  
    code  
    ..  
    .  
    return resultat
```

Exemple

```
## exemple:  
def myf(x):  
    y = x+5  
    return y  
  
## appel de la fonction pour l'utiliser  
t = myf(5)
```

Note :

- On peut faire une **fonction qui ne retourne rien** (ce qui est appelé en algorithmique : **une procédure**)

Syntaxe en algorithmique

```
fonction nom_de_la_fonction(paramètres)  
    instructions  
    retourner valeur_de_retour  
fin fonction
```

Exemple

```
fonction calculerSomme(a, b)  
    somme = a + b  
    retourner somme  
fin fonction  
  
début  
    // Appel de la fonction calculerSomme avec les paramètres 3 et 5  
    résultat = calculerSomme(3, 5)  
    afficher "La somme est égale à ", résultat  
fin
```

Fonction VS Procédure

- En programmation, les termes "fonction" et "procédure" sont souvent utilisés pour désigner des blocs de code réutilisables. Cependant, il existe une petite différence entre les deux.

1. Fonction :

- Une fonction est un bloc de code qui retourne une valeur.
- Elle peut prendre des paramètres en entrée et renvoie un résultat.
- La valeur de retour peut être utilisée dans d'autres parties du code.

```
► def addition(a, b):  
    result = a + b  
    return result  
  
# Utilisation de la fonction  
somme = addition(3, 4)  
print(somme)
```

7

2. Procédure :

- Une procédure est également un bloc de code réutilisable, mais elle n'a pas de valeur de retour explicite.
- Elle peut effectuer des actions, mais ne produit pas de résultat que vous pouvez utiliser ailleurs dans le code.

```
► def afficher_message(message):  
    print("Message :", message)  
  
# Utilisation de la procédure  
afficher_message("Bonjour, monde!")  
  
Message : Bonjour, monde!
```

Arguments positionnels
Arguments par mot-clé
Arguments par défaut
et Argument à nom forcé

1. **Arguments Positionnels** : Les arguments positionnels sont passés à une fonction dans l'ordre dans lequel les paramètres sont définis dans la signature de la fonction. C'est la méthode la plus courante.

```
➤ def afficher_informations(nom, age):  
    print("Nom:", nom)  
    print("Âge:", age)  
  
# Appel avec des arguments positionnels  
afficher_informations("Alice", 25)
```

Nom: Alice
Âge: 25

2. **Arguments par Mot-Clé** : Les arguments par mot-clé sont spécifiés en utilisant le nom du paramètre suivi de la valeur, ce qui permet de passer les arguments dans un ordre différent de celui de la signature de la fonction.

```
➤ def afficher_informations(nom, age):  
    print("Nom:", nom)  
    print("Âge:", age)  
  
# Appel avec des arguments par mot-clé  
afficher_informations(age=30, nom="Bob")
```

Nom: Bob
Âge: 30

Lors de l'appel d'une fonction, vous pouvez également mélanger les deux types d'arguments

```
► def afficher_informations(nom, age, ville):  
    print("Nom:", nom)  
    print("Âge:", age)  
    print("Ville:", ville)  
  
# Appel avec des arguments positionnels et par mot-clé  
afficher_informations("Charlie", ville="Paris", age=22)
```

Nom: Charlie
Âge: 22
Ville: Paris

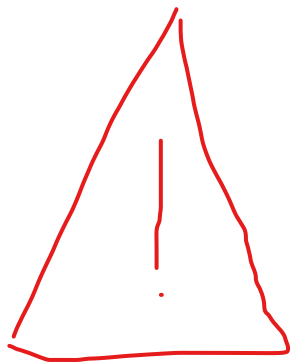
► *## Mais, Ne pas donner les arguments mot-clés en premier, ça va donner une erreur !!!!*

```
afficher_informations(nom="Charlie", "Paris", 22)
```

Cell In[26], line 3

```
    afficher_informations(nom="Charlie", "Paris", 22)  
                        ^
```

SyntaxError: positional argument follows keyword argument



Argument par défaut

Argument fixe ou par défaut:

```
def surface(long=2, larg=2):  
    s = long * larg  
    return s
```

```
surface()    ## je peux l'utiliser sans donner aucun argument  
            ## si je donne aucun argument, il utilise les valeurs par défaut
```

4

```
surface(5, 8)    ## et je peux l'utiliser avec d'autres valeurs
```

40

```
surface(5)       ## ou modifier juste une parmi les deux
```

10

autre exemple (fonction print de Python)

```
print("bonjour", "fssm")    ## *objects, sep=' ', end='\n'  
                            ## ce qui est nécessaire de donner en argument, c'est la valeur à afficher  
                            ## Le séparateur ou la fin de l'affichage ont déjà des valeurs par défaut
```

bonjour fssm

```
print("bonjour" , "fssm", sep=' ---->>> ', end='\n\n\n\n\n\n')
```

bonjour ---->>> fssm

Argument à nom forcé

```
## comment Python force l'utilisateur de spécifier l'argument séparateur dans la fonction print par nom ?  
## ----->>> en rajoutant le symbole *  
## toute argument qui vient après '*' est un argument à donner par nom
```

```
def exemple(pos, *, forcé):  
    print(pos, forcé)
```

```
exemple(1, forcé=12)
```

```
1 12
```

```
## essayer sans nom --> erreur  
exemple(1, 12)
```

TypeError

Traceback (most recent call last)

Cell In[18], line 1

----> 1 exemple(1, 12)

TypeError: exemple() takes 1 positional argument but 2 were given

Argument à nom forcé

```
## aussi, vous pouvez utiliser de la manière suivante comme la fonction print de Python :  
## *valeurs prend une succession de valeurs comme un tuple  
  
def exemple(pos, *valeurs, forcé):  
    print(pos, valeurs, forcé)           #ou *valeurs ici aussi pour ouvrir le tuple  
  
exemple(1, 10, 20, 30, 40, forcé=55)    ## forcé doit obligatoirement être spécifié par nom,  
                                         ## parcequ'on sait pas la taille de l'argument valeurs
```

```
1 (10, 20, 30, 40) 55
```

```
## une autre approche **valeurs:  
## qui permet de prendre une succession d'arguments avec leurs noms comme étant un dictionnaire  
  
def exemple(**valeurs):  
    for clé, valeur in valeurs.items():  
        print(clé, valeur)  
  
exemple(a=1, b=2, c=3)
```

```
a 1  
b 2  
c 3
```

Argument à nom forcé

Attention lors de l'utilisation de ****valeurs** avec d'autres arguments !!

```
## utilisation avec d'autre arguments
```

```
def exemple(nom, faculté, **valeurs):  
    print(nom, faculté)  
    for clé, valeur in valeurs.items():  
        print(clé, valeur)  
  
exemple("Mohamed", "FSSM", a=1, b=2, c=3)
```

```
Mohamed FSSM  
a 1  
b 2  
c 3
```



```
## Attention, toujours **valeurs doit être le dernier argument  
## Le contraire générera une erreur de syntaxe
```

```
def exemple(**valeurs, nom, faculté):  
    for clé, valeur in valeurs.items():  
        print(clé, valeur)  
  
    print(nom, faculté)  
  
exemple(a=1, b=2, c=3, nom="Mohamed", faculté="FSSM")
```

```
Cell In[43], line 4
```

```
def exemple(**valeurs, nom, faculté):
```

```
SyntaxError: invalid syntax
```



Variables locales et Variables globales

Les variables locales et globales sont deux types de variables que l'on peut rencontrer lors de la programmation en utilisant des fonctions. Comprendre la portée (scope) de ces variables est crucial pour éviter des erreurs potentielles et écrire un code propre et bien structuré.

Variables Locales :

- Portée : Les variables locales sont définies à l'intérieur d'une fonction et ne sont accessibles qu'à l'intérieur de cette fonction.
- Durée de vie : Elles existent tant que la fonction s'exécute et sont détruites dès que la fonction se termine.

Variables Globales :

- Portée : Les variables globales sont définies en dehors de toutes les fonctions et sont accessibles partout dans le programme.
- Durée de vie : Elles existent pendant toute la durée d'exécution du programme.

Principes d'utilisation :

- Les variables locales sont généralement préférées car elles limitent la portée des variables aux parties spécifiques du code où elles sont nécessaires, évitant ainsi les interférences involontaires.
- Les variables globales peuvent être utiles lorsque vous avez besoin d'une variable qui doit être partagée entre plusieurs fonctions ou qui doit rester constante tout au long du programme.
- Évitez d'utiliser des variables globales lorsque des variables locales suffisent, car cela peut rendre votre code moins lisible et plus sujet à des erreurs difficiles à traquer.

Variable **locale**:

Exemple sous Python

```
► def fonction_locale():  
    x = 10 # Variable locale  
    print("À l'intérieur de la fonction locale, x =", x)  
  
# Appel de la fonction  
fonction_locale()
```

À l'intérieur de la fonction locale, x = 10

```
► # Tentative d'accéder à la variable locale en dehors de la fonction (cette ligne générera une erreur)  
print("la valeur de x est", x)
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[29], line 2  
      1 # Tentative d'accéder à la variable locale en dehors de la fonction (cette ligne générera une erreur)  
----> 2 print("En dehors de la fonction, x =", x)  
  
NameError: name 'x' is not defined
```

Variable globale:

Exemple sous Python

```
## si on veut juste afficher une variable globale dans une fonction  
## ça va marcher
```

```
y = 20
```

```
def fonction_globale():  
    print("À l'intérieur de la fonction globale, y =", y)
```

```
fonction_globale()
```

```
# Accès à la variable globale en dehors de la fonction  
print("la valeur de y est", y)
```

À l'intérieur de la fonction globale, y = 20
la valeur de y est 20

```
## ou même affichage modifié
```

```
y = 20
```

```
def fonction_globale():  
    print("À l'intérieur de la fonction globale, y =", y+5)
```

```
fonction_globale()
```

```
# Accès à la variable globale en dehors de la fonction  
print("la valeur de y est", y)
```

À l'intérieur de la fonction globale, y = 25
la valeur de y est 20

Variable globale:

Exemple sous Python (attention erreur fréquent !!)

```
## mais pas, avec modification qui crée une autre variable
```

```
y = 20
```

```
def fonction_globale():
```

```
    y = y + 5
```

```
    print("À l'intérieur de la fonction globale, y =", y)
```

```
fonction_globale()
```

```
-----  
UnboundLocalError
```

```
Traceback (most recent call last)
```

```
Cell In[22], line 9
```

```
6     y = y + 5
```

```
7     print("À l'intérieur de la fonction globale, y =", y)
```

```
----> 9 fonction_globale()
```

```
Cell In[22], line 6, in fonction_globale()
```

```
5 def fonction_globale():
```

```
----> 6     y = y + 5
```

```
7     print("À l'intérieur de la fonction globale, y =", y)
```

```
UnboundLocalError: local variable 'y' referenced before assignment
```

```
## pourquoi, parceque ""y = y + 5"" va créer une autre variable y qui est locale  
## qui n'a aucune valeur jusqu'à présent, et puis on veut l'affecter ""y + 5"" ,  
## c'est pourquoi on a l'erreur: local variable 'y' referenced before assignment
```

```
a = 1
```

```
id(a)
```

```
1372409102576
```

```
a = a+1
```

```
id(a)
```

```
1372409102608
```

```
## vous remarquer ici que les variables (primitives) sont immutables  
## pour les changer on crée une autre variable
```

Variable **globale**:

Solution pour le problème précédent

Utilisation du mot-clé prédéfini : **global**

```
y = 20

def fonction_globale():
    global y # Utilisation du mot-clé global pour indiquer que nous faisons référence à la variable globale
    y = y + 5
    print("À l'intérieur de la fonction globale, y =", y)

fonction_globale()

# Accès à la variable globale en dehors de la fonction
print("la valeur de y est", y)
```

À l'intérieur de la fonction globale, y = 25
la valeur de y est 25